

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 17: TIPOS DERIVADOS EM MPI

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 22/09/2026.

1. EXPERIMENTO

Reimplemente a tarefa 16, agora distribuindo as colunas entre os processos. Utilize MPI_Type_vector e MPI_Type_create_resized para definir um tipo derivado que represente colunas da matriz. Use MPI_Scatter com esse tipo para distribuir blocos de colunas, e MPI_Scatter ou cópia manual para enviar os segmentos correspondentes de x. Cada processo deve calcular uma contribuição parcial para todos os elementos de y e usar MPI_Reduce com MPI_SUM para somar os vetores parciais no processo 0. Discuta as diferenças de acesso à memória e desempenho em relação à distribuição por linhas.

2. CONCEITOS RELEVANTES

- **Matriz em ordem de linhas (row-major):** em C, uma matriz A[M][N] é armazenada linha após linha. Elementos consecutivos de uma linha ficam próximos na memória, elementos de uma coluna ficam separados por N posições.
- **Localidade de memória e cache:** percorrer dados contíguos costuma ser mais rápido porque o processador traz blocos próximos para a cache. Por isso, dividir a matriz por linhas favorece o acesso à memória.

- **Tipo derivado MPI:** MPI_Type_vector permite descrever dados não contíguos, como partes de colunas de uma matriz armazenada por linhas. Ele evita copiar manualmente cada coluna antes de enviá-la.
- **MPI_Type_create_resized:** ajusta a extensão de um tipo derivado. Neste caso, ele faz com que cada envio pelo MPI_Scatter avance exatamente um bloco de colunas, em vez de avançar pelo espaço total ocupado pelo tipo com seus saltos.
- **MPI_Gather versus MPI_Reduce:** na divisão por linhas, cada processo calcula posições diferentes de y, então MPI_Gather reúne partes distintas. Na divisão por colunas, todos calculam contribuições para as mesmas posições de y, então é necessário somá-las com MPI_Reduce.
- **Escalabilidade:** usar mais processos não garante menor tempo. Para matrizes pequenas, o custo de comunicação, sincronização e criação de processos pode ser maior que o ganho do paralelismo.

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador AMD Ryzen 7 5700X (8 núcleos físicos e 16 threads). O sistema dispõe também de 32 GB de memória RAM e um SSD de 1TB.

4. CÓDIGO

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

static void preencherDados(double *matriz, double *vetor, int linhas, int colunas) {
    for (int i = 0; i < linhas * colunas; i++) {
        matriz[i] = (double)((i % 10) + 1);
    }

    for (int i = 0; i < colunas; i++) {
        vetor[i] = 1.0;
    }
}
```

```c
int main(int argc, char **argv) {
  MPI_Init(&argc, &argv);

  int processo, quantidadeProcessos;
  MPI_Comm_rank(MPI_COMM_WORLD, &processo);
  MPI_Comm_size(MPI_COMM_WORLD, &quantidadeProcessos);

  int linhas = 1000;
  int colunas = 1000;

  if (argc == 3) {
    linhas = atoi(argv[1]);
    colunas = atoi(argv[2]);
  } else if (argc != 1) {
    if (processo == 0) {
      fprintf(stderr, "Uso: %s [linhas colunas]\n", argv[0]);
    }
    MPI_Finalize();
    return EXIT_FAILURE;
  }

  if (linhas <= 0 || colunas <= 0 || colunas % quantidadeProcessos != 0) {
    if (processo == 0) {
      fprintf(stderr,
              "As dimensoes devem ser positivas e o numero de colunas "
              "deve ser divisivel pelo numero de processos.\n");
    }
    MPI_Finalize();
    return EXIT_FAILURE;
  }

  // Cada processo recebera a mesma quantidade de colunas da matriz A.
  const int colunasLocais = colunas / quantidadeProcessos;
  double *matriz = NULL;
  double *vetor = NULL;
  double *resultado = NULL;
  double *matrizLocal = malloc((size_t)linhas * colunasLocais * sizeof(*matrizLocal));
  double *vetorLocal = malloc((size_t)colunasLocais * sizeof(*vetorLocal));
```

```c
double *resultadoParcial = calloc((size_t)linhas, sizeof(*resultadoParcial));

if (matrizLocal == NULL || vetorLocal == NULL || resultadoParcial == NULL) {
    fprintf(stderr, "Processo %d: falha ao alocar memoria.\n", processo);
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
}

// Somente o processo 0 cria os dados completos de A, x e y.
if (processo == 0) {
    matriz = malloc((size_t)linhas * colunas * sizeof(*matriz));
    vetor = malloc((size_t)colunas * sizeof(*vetor));
    resultado = malloc((size_t)linhas * sizeof(*resultado));
    if (matriz == NULL || vetor == NULL || resultado == NULL) {
        fprintf(stderr, "Falha ao alocar memoria no processo 0.\n");
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    }
    preencherDados(matriz, vetor, linhas, colunas);
}

/*
 * A matriz e armazenada por linhas em C. Para acessar o mesmo bloco de
 * colunas na proxima linha, e necessario saltar "colunas" elementos.
 * MPI_Type_vector descreve esses blocos nao contiguos na matriz original.
 */
MPI_Datatype blocoColunas, blocoColunasRedimensionado;
MPI_Type_vector(
    linhas, // numero de blocos
    colunasLocais, // numero de elementos em cada bloco
    colunas, // numero de elementos entre o inicio de cada bloco
    MPI_DOUBLE, // tipo de dado
    &blocoColunas // tipo de dado resultante
);
// cria um tipo redimensionado para que o deslocamento entre blocos seja correto
MPI_Type_create_resized(
    blocoColunas, // tipo de dado original
    0, // deslocamento inicial
    (MPI_Aint)colunasLocais * sizeof(double), // tamanho do bloco
    &blocoColunasRedimensionado // tipo de dado resultante
);
```

```
// comita o tipo de dado para que possa ser usado em comunicacoes
MPI_Type_commit(&blocoColunasRedimensionado);
MPI_Type_free(&blocoColunas);

MPI_Barrier(MPI_COMM_WORLD);
const double inicio = MPI_Wtime();

/*
 * O tipo redimensionado faz MPI_Scatter avancar um bloco de colunas por
 * vez. No receptor, o bloco chega de forma contigua em matrizLocal.
 */
MPI_Scatter(matriz, 1, blocoColunasRedimensionado,
            matrizLocal, linhas * colunasLocais, MPI_DOUBLE,
            0, MPI_COMM_WORLD);

// Cada processo recebe os elementos de x associados as suas colunas.
MPI_Scatter(vetor, colunasLocais, MPI_DOUBLE,
            vetorLocal, colunasLocais, MPI_DOUBLE,
            0, MPI_COMM_WORLD);

// Cada processo calcula uma contribuicao parcial para todas as linhas de y.
for (int i = 0; i < linhas; i++) {
    for (int j = 0; j < colunasLocais; j++) {
        resultadoParcial[i] += matrizLocal[i * colunasLocais + j] * vetorLocal[j];
    }
}

// MPI_SUM soma as contribuicoes parciais e deixa o vetor y completo no processo 0.
MPI_Reduce(
    resultadoParcial,
    resultado,
    linhas,
    MPI_DOUBLE,
    MPI_SUM,
    0,
    MPI_COMM_WORLD
);

if (processo == 0) {
```

```
    const double tempo = MPI_Wtime() - inicio;
    printf("M=%d N=%d processos=%d tempo_s=%.6f y0=%.1f\n",
           linhas, colunas, quantidadeProcessos, tempo, resultado[0]);
  }

  MPI_Type_free(&blocoColunasRedimensionado);
  free(matriz);
  free(vetor);
  free(resultado);
  free(matrizLocal);
  free(vetorLocal);
  free(resultadoParcial);

  MPI_Finalize();
  return EXIT_SUCCESS;
}
```

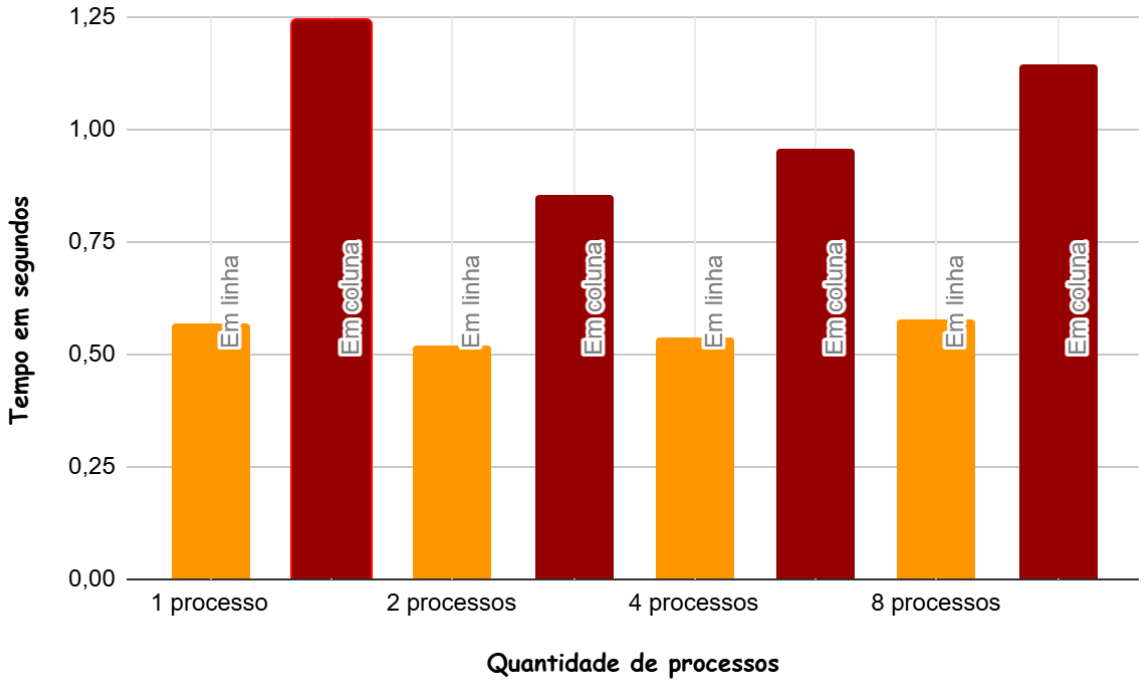
5. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho do experimento.

Tabela 1: Comparação de desempenho de distribuição com colunas e linhas (tarefa 16)

	1 processo		2 processos		4 processos		8 processos	
Matriz	**Linha**	**Coluna**	**Linha**	**Coluna**	**Linha**	**Coluna**	**Linha**	**Coluna**
512 × 512	**0,000446**	0,001128	**0,00047**	0,0009335	**0,000263**	0,000765	**0,000193**	0,000840
1000 × 1000	**0,001121**	0,002588	**0,001386**	0,0027405	**0,00149**	0,003116	**0,00068**	0,001728
4000 × 4000	**0,014214**	0,03104	**0,013023**	0,024942	**0,013406**	0,0265125	**0,015538**	0,033884
8000 × 8000	**0,05436**	0,124552	**0,047417**	0,0967035	**0,048169**	0,093426	**0,052347**	0,104898
12000 × 12000	**0,122185**	0,307627	**0,104622**	0,250229	**0,114765**	0,234859	**0,120764**	0,298707
16000 × 16000	**0,217347**	0,528825	**0,247048**	0,476248	**0,193111**	0,399086	**0,262676**	0,560698
20000 × 20000	**0,377424**	0,913399	**0,328802**	0,745219	**0,356001**	0,750068	**0,38947**	0,877288
24000 × 24000	**0,571196**	1,24233	**0,521082**	0,85354	**0,538986**	0,956019	**0,577285**	1,14666

Figura 2: Comparação de desempenho de distribuição com colunas e linhas (tarefa 16) na matriz 24000 x 240000



Para entender os resultados, é importante perceber que na distribuição por linhas, cada processo recebe um conjunto completo de linhas da matriz. Como a matriz está armazenada em memória linha por linha, os dados enviados para cada processo são contínuos. Isso é vantajoso porque o acesso à memória ocorre de forma sequencial, aproveitando melhor a cache do processador.

Além disso, cada processo calcula apenas as posições de “y” correspondentes às suas linhas. No final, basta usar “MPI_Gather” para juntar os pedaços do vetor resultado no processo 0. Dessa forma, cada processo envia somente uma parte de “y”.

Na distribuição por colunas, cada processo recebe algumas colunas da matriz e a parte correspondente do vetor “x”. O problema é que, em uma matriz armazenada por linhas, os elementos de uma coluna ficam separados na memória. Por exemplo, para acessar dois elementos consecutivos de uma mesma coluna, é necessário saltar o tamanho total da linha.

Por isso, foi necessário usar “MPI_Type_vector” para descrever os elementos espaçados de cada coluna e “MPI_Type_create_resized” para permitir que o “MPI_Scatter” envie blocos de colunas corretamente. Embora isso facilite a

distribuição, o acesso inicial aos dados não é tão eficiente quanto o envio de blocos de linhas contínuos.

Depois da distribuição, cada processo calcula uma contribuição parcial para todas as posições de “y”. Por exemplo, se houver quatro processos, todos calculam um vetor parcial de tamanho “M”. Ao final, esses vetores precisam ser somados com “MPI_Reduce” usando “MPI_SUM”.

Essa etapa também aumenta o custo de comunicação: na versão por linhas, os processos apenas enviam partes diferentes de “y”; na versão por colunas, todos precisam participar da soma de um vetor completo de tamanho “M”.

Os resultados confirmam essa diferença. Para a matriz “24000 × 24000”, a distribuição por linhas levou cerca de “0,57s” com um processo, enquanto a versão por colunas levou “1,24s”. Com mais processos, a versão por colunas continua apresentando tempos maiores na maioria dos casos.

Portanto, para matrizes armazenadas por linhas, a divisão por linhas tende a ter melhor desempenho. Ela aproveita melhor a organização da memória, distribui dados contíguos e exige menos comunicação na montagem do vetor resultado.